

A Brief Guide to Autoformalization in Mathematical Physics

James Squires

May 27, 2026

1 Type Theoretic Formal Verification

Recent developments in computer science have enabled mathematicians to code mathematical proofs with verified correctness. This stems from the development of Interactive Theorem Provers, functional coding languages that enable the user to do type-checked math. We can combine these coding languages with AI to automatically prove theorems. Doing so gives the computational speed and linguistic capabilities of Large Language Models (LLMs) along with the ability to verify LLM generated statements for mathematical correctness. While typically used in mathematics, this neurosymbolic framework can also be leveraged in mathematical physics settings.

1.1 Proofs

We can think of mathematical proofs in terms of *proof trees*. Given a statement, there are certain actions that can *derive* other possible statements; these actions are *derivation rules*. As theorems are proven, we can add to the list of derivation rules.

1.2 Proofs as Types

Mathematical logic is in one-to-one correspondence with what is known as *type theory* (see [5] for more on this). Type theory is the programmatic language for mathematical logic¹ This correspondence enables the *coding* of mathematics.

The jist of the correspondence goes as follows: for any theorem we derive, there corresponds a type. And for any proof of this theorem, there corresponds a term that inhabits this type. If the mathematician can define their theorem and then show that a proof inhabits this theorem as a term, then they have succeeded in proving the theorem.

Examples of types that you may be familiar with are: strings, floats, booleans, natural numbers, or integers. Terms that inhabit these types are ‘cat’, ‘2.01’, ‘True’, ‘3’, and ‘-1’ respectively. Additionally, in Interactive Theorem Provers such as Lean (this is the favored coding language for neurosymbolic AI right now), we might also have types like: Prop or the theorem ‘2 is in the natural numbers’. The theorem is actually a term of the type Prop (short for proposition). This is because in Lean, types are terms of other types. Our Prop type has the type called Type, and Type has the type called Type1, Type1 has the type Type2 and so on.... These are the different universes of Lean. Think of this as being similar to how we define sets, classes, and higher-order collections in ZFC axioms.

When we provide a proof in type theory (or standard logic), we can think of the type *derivation* in terms of proof trees as mentioned in Section 1.1. This allows us to formulate mathematical proofs as search problems. The next section will expound on this.

¹For the sake of accuracy, this isn’t just any logic, but a form of constructive logic known as the BHK-Interpretation. The BHK-Interpretation is also referred to as *intuitionism* or *constructivism* (the difference is subtle).

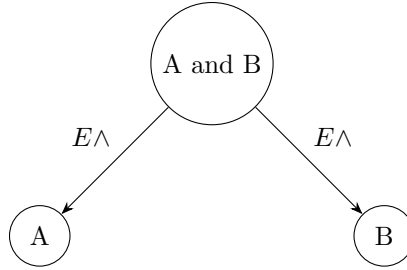


Figure 1: A derivation tree showing how we can obtain proposition A and proposition B from the proposition A and B. $E\wedge$ is the *elimination-and* rule; this is a rule defined in logic that does exactly what is shown in this figure.

1.3 Search Problems

A search problem can be evaluated computationally if we create a *tree* where the nodes are *states* and the directed edges are *actions*. In the context of mathematical theorem proving, states are theorem statements, and actions are derivation rules. Quite often mathematics produces incredibly large trees; this poses challenges for finding a particular theorem.

1.4 Formal Verification of Proofs

When we make derivation trees, because type theory has a mechanism for evaluating whether a derivation is *well-typed*, we can *type check* our proofs. This is a stronger version of what you might have seen in C++ or Java where type-checking is performed upon compilation. Type checking allow us to guarantee *soundness* or *correctness* of our proofs (C++ is *partially-typed*, whereas Lean is *dependently-typed*).

Another good example of type-checking for those who haven't coded much is dimensional analysis². When a physicist solves for velocity, they keep track of the physical units their values have. For example, a distance has units of meters. If a solution for velocity is in units of meters squared, this implies something went wrong with the calculation, as the answer should be in some equivalent form of meters per second. Just like how a physicist or chemist checks their units, a computer checks types. When there is a unit error, the physicist backtracks. And when there is a type-check error, the computer scientist knows something in the code went wrong.

2 Using AI to Automatically Prove Theorems

2.1 AI as a Search Algorithm

AI forms a search algorithm on our proof trees. This is the power of LLMs—they form extremely good search algorithms for mathematical proofs which are otherwise infinitely large search-spaces.

When the AI performs a search, it decides what search *action* it will take by generating what is called a *tactic*. This is essentially just one mathematical step for a Lean proof. If this action fails (if the tactic throws an error or takes the proof further from being finished), then the AI decides to backtrack and attempt another action and so on. More sophisticated AI powered automated theorem provers might also do things like parse theorems into smaller lemmas etc. AI automated theorem proving has largely had success with formalizations in category theory, euclidean geometry, algebra, etc. See [2, 8, 9]. As I am writing this, many more break throughs are appearing in various other fields such as discrete geometry and number theory.

²You can also think of dimensions as abelian groups. Along this line of thinking, a $\mathbb{Z}^7$ graded Laurent polynomial encodes the arithmetic of physics.

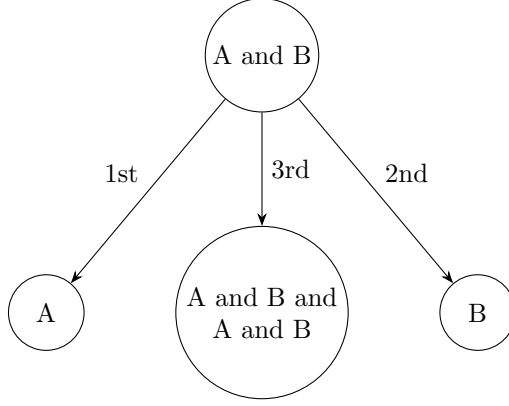


Figure 2: There are several actions our search can take. This figure would demonstrate a proof search for something like "Give proposition A using the fact that we have A and B". The LLM generates responses that first attempt the action for proposition A, then B, then it might do something like introduce A and B (the other actions are terrible; AI would certainly not struggle with this problem).

2.2 Limitations

Once we have defined a theorem we want proved, then we can use AI interfaced with these formal verification codes to perform the search problem. This is because, once axioms, definitions, and theorems are defined, we have a closed system that we can search through. This is the *autoformalization* of a proof. What is much harder is to have AI formulate and define the correct theorem we want to prove (without any semantic drift). This is known as the *specification-problem*. While AI cannot mislead the user with *autoformalization* (a proof is either correct or unfinished), it can mislead the user by misspecifying definitions or theorems. For the practicing physicist, this is arguably the biggest take away: formal specification should always be looked over. In fact, I encourage the user to learn how to define the problems and definitions themselves, and largely stick to only autoformalization (this is the most responsible form of automated theorem proving).

3 Automated Theorem Proving in Physics

I suggest several applications for automated theorem proving in physics in a practical setting:

1. *Checking mathematical derivaitons*
2. *Formal verification/checking of work*
3. *Searching for unsolved solutions (hail marrys)*

Checking mathematical derivations should be done as an extra, supplementary step. This way, the user reaps all the benefits of powerful neurosymbolic AI models without none of the downsides.

Formal verification of work in a high-stakes setting should not be done with automated theorem provers if part of the AI implemented work is formal specification. You can't rely on vibe-coded formal specifications like you would a proof, as there is no way of truly formally verifying specification.

The hail marry solution is fairly practical. Don't let this be the research, but rather, you can use this to propel your research and motivate intermediate steps. AI is suprisingly good at providing creative solutions in this domain.

3.1 An Example of Automated Theorem Proving

I used Aristotle to formally specify and prove the iconic No-Cloning theorem from quantum computing as seen in the figure below. Lean syntax may appear incoherent to the uninitiated, so I have included a

translation on the right³:

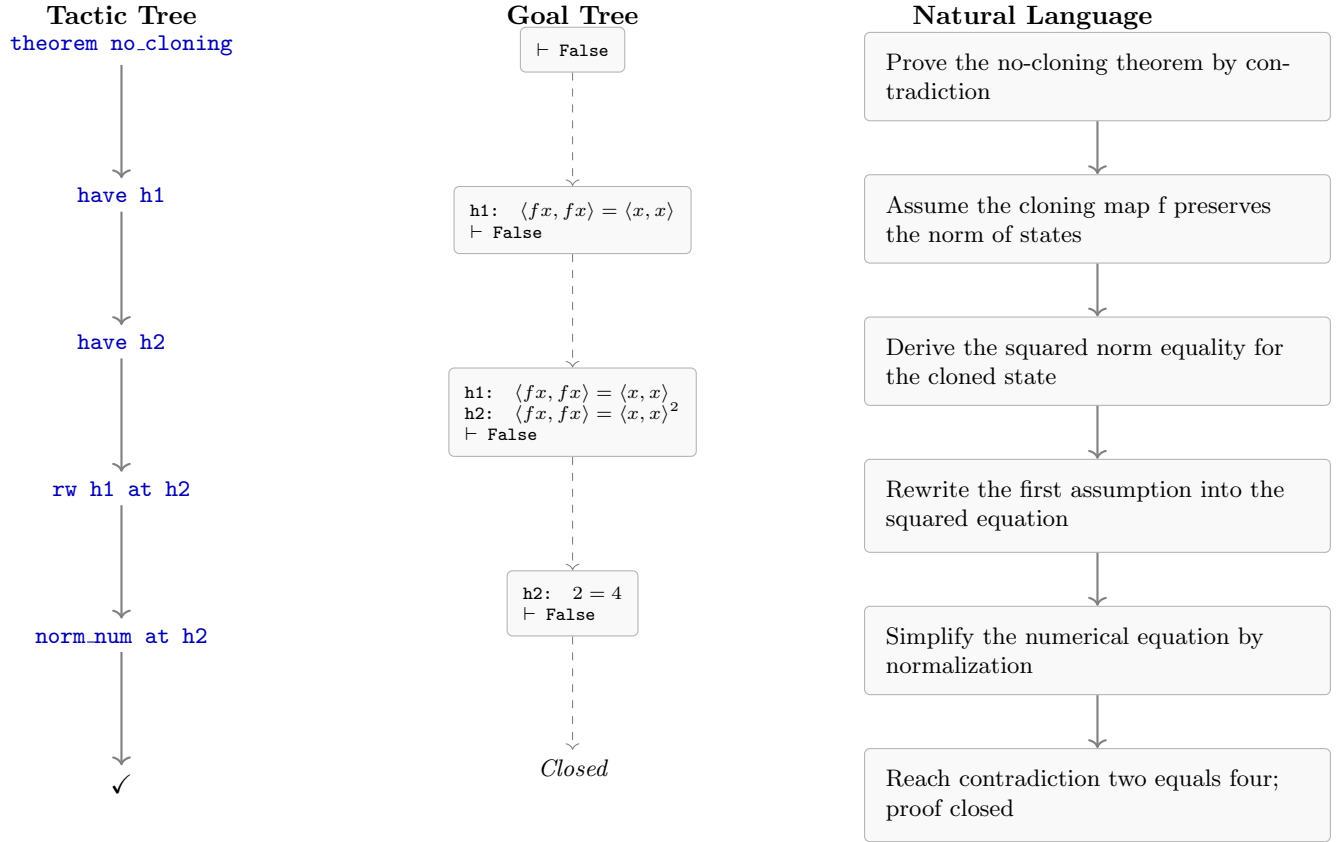


Figure 3: Final Proof Tree of No-Cloning Theorem. The far-left column corresponds to the actual tactics Aristotle wrote. The middle column corresponds to how the goal-state changed in response to these tactics, where the goal-state is what we have left to prove. The right side is a natural language translation of this proof.

3.2 PhysLean

Currently there is a small yet concerted effort to formalize physics in Lean. The repository is called PhysLean [1]. The community running PhysLean is truly incredible and I encourage anyone interested in checking them out.

3.3 Other Physics in Lean

For some reference, I'm also currently coding categorical quantum mechanics in Lean. I've linked my Github repo here [6]. If you're interested, please reach out.

³The Lean code is also slightly simplified.

4 A Note on Barrier to Entry and Resources

The software this exposition has mentioned is incredible. However, I feel it is my duty to warn the reader that the barrier to entry for understanding many of these concepts is quite high. I do not say this as a gatekeeper, but rather as someone who has suffered through the learning process for the past year and a half. My hope is that type theory, Lean, and automated theorem proving becomes far more accessible to working scientists outside of theoretical computer science. In the mean time, there are many approaches the reader can take to understanding Lean and how AI works.

4.1 Resources for Lean and Type Theory

There are server games for learning Lean found here. I recommend trying these, but for those with little functional programming experience, this might even be a lot. For those with some mathematical maturity, I recommend checking out *A Tutorial Introduction to the Lambda Calculus* by Raul Rojas [4] (this would be the equivalent of learning set theory to then learn analysis). To fully embrace the typed mindset, the best resource I’ve found is *Type Theory and Formal Proof: An Introduction* by Nederpelt and Guevers [3]. And if you’re all in, I recommend *Lectures on the Curry–Howard Isomorphism* [5].

4.2 Resources for Automated Theorem Proving

As far as exploring AI capabilities, I will point to two resources. Researchers recently came out with *Aristotle*, an AI model that converts natural language proofs (what we typically think of when we do proofs) to Lean proofs [7]. Aristotle is incredibly user-friendly and I encourage all readers to check it out [7]. I’ve personally used Aristotle to verify my work as an extra layer of safety. Another recent development is *LeanDojo*, a software you can use to iteratively prove theorems with AI [9]. The software setup is a bit more technical, but highly rewarding.

References

- [1] The Physlib community. *Physlib: The Lean Physics Library*. <https://github.com/leanprover-community/physlib>. 2024.
- [2] Logan Murphy et al. “Autoformalizing Euclidean Geometry”. In: (2024). arXiv preprint.
- [3] Rob Nederpelt and Herman Geuvers. *Type Theory and Formal Proof: An Introduction*. Cambridge University Press, 2014. ISBN: 978-1-107-03650-5. DOI: 10.1017/CB09781139567725. URL: <https://doi.org/10.1017/CB09781139567725>.
- [4] Raúl Rojas. “A Tutorial Introduction to the Lambda Calculus”. In: *arXiv preprint arXiv:1503.09060* (2015). arXiv: 1503.09060 [cs.LO]. URL: <https://arxiv.org/abs/1503.09060>.
- [5] Morten Heine Sørensen and Paweł Urzyczyn. *Lectures on the Curry–Howard Isomorphism*. Vol. 149. Studies in Logic and the Foundations of Mathematics. Amsterdam: Elsevier Science, 2006. ISBN: 9780444520777. URL: <https://www.sciencedirect.com/book/9780444520777/lectures-on-the-curry-howard-isomorphism>.
- [6] James Squires. *Lean 4 Formal Verification of Abramsky-Coecke Semantics*. 2026. URL: https://github.com/JamesSquires0/Abramsky_Coecke_Lean/tree/main.
- [7] The Harmonic Team. “Aristotle: IMO-level Automated Theorem Proving”. In: (2025). arXiv preprint. URL: <https://arxiv.org/pdf/2510.01346>.
- [8] Rongge Xu et al. “LeanCat: A Benchmark Suite for Formal Category Theory in Lean (Part I: 1-Categories)”. In: (2025). arXiv preprint. URL: <https://github.com/sciencraft/LeanCat>.
- [9] Kaiyu Yang et al. “LeanDojo: Theorem Proving with Retrieval-Augmented Language Models”. In: (2023). arXiv preprint. URL: <https://leandojo.org>.